

Computer Architecture: *Memory Busses*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store data?
- **How does the machine interface using buses?**
- What does it mean to have “allowed” memory?
- How does the machine speed up data access?

First Draft of a Report
on the EDVAC

by
John von Neumann

Contract No. W-670-ORD-4926

Between the
United States Army Ordnance Department
and the
University of Pennsylvania

Moore School of Electrical Engineering
University of Pennsylvania

June 30, 1945

This is an exact copy of the original typescript draft as obtained from the University of Pennsylvania Moore School Library except that a large number of typographical errors have been corrected and some missing information has been filled in as provided where possible. Missing

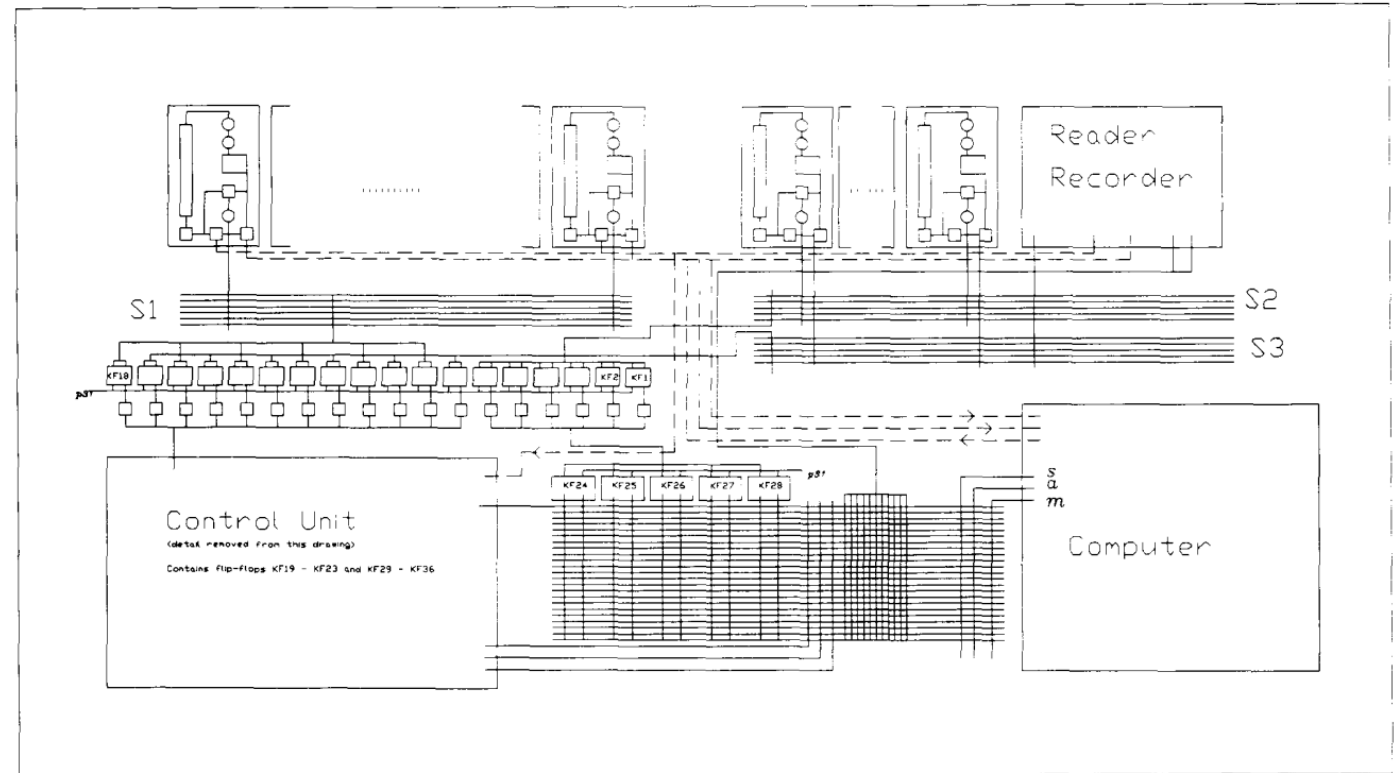
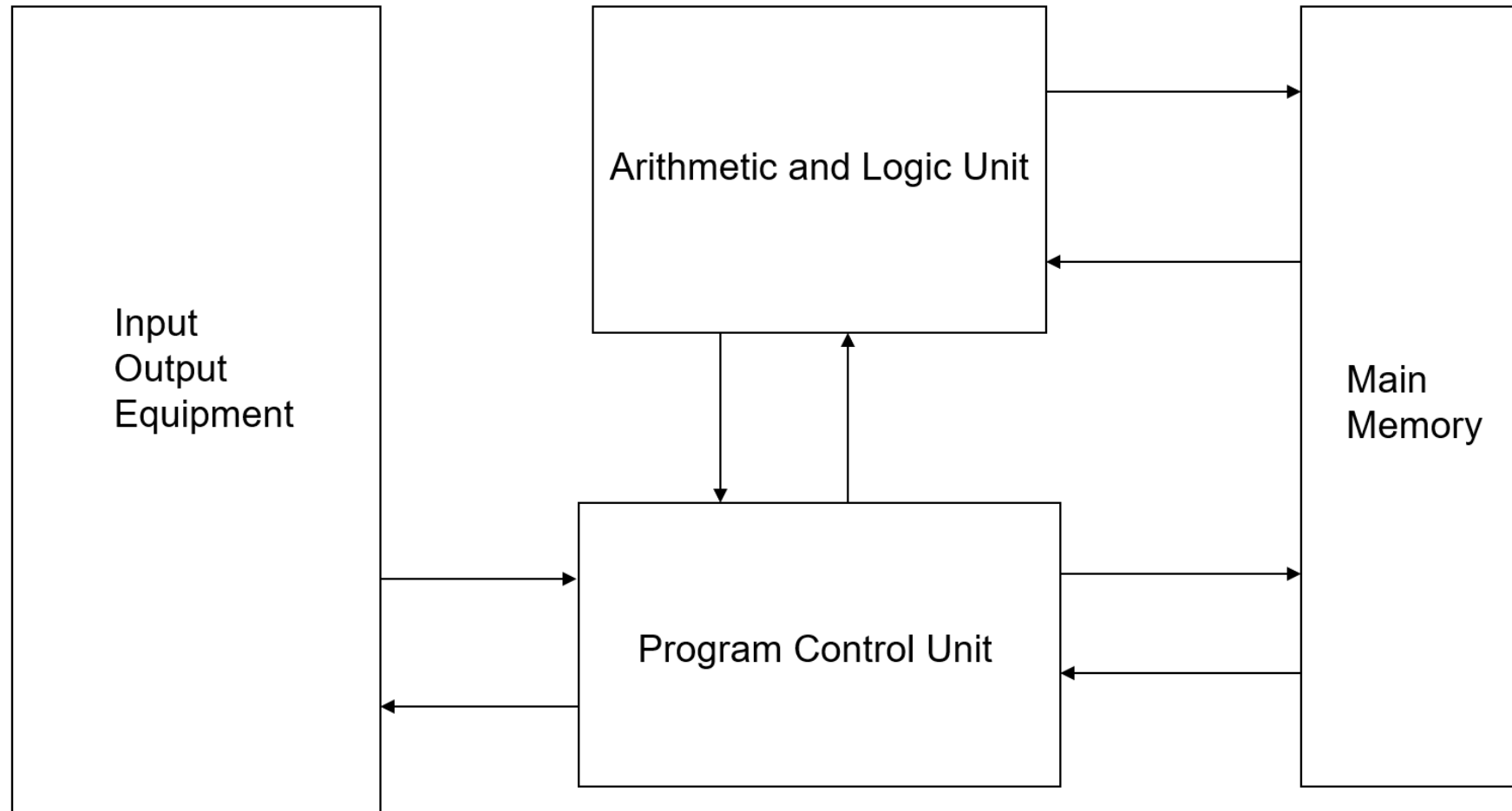


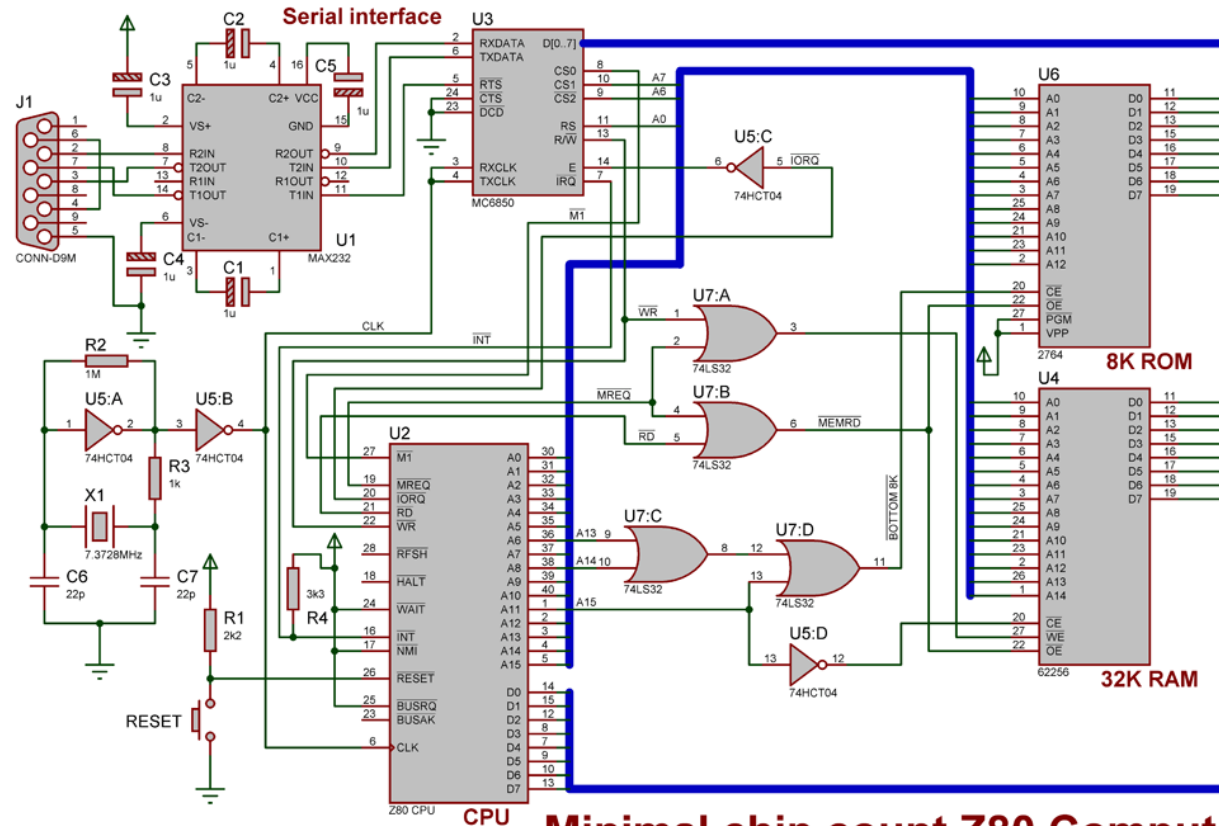
Figure 2. The August 1946 design for a "Serial Acoustic Binary EDVAC," as it was shown to students during the Moore School lectures (redrawn from the original with some detail removed to avoid clutter).

<https://web2.qatar.cmu.edu/~mhhammou/15346-s13/resources/OriginsOfComputers/Origins%20Fate%20of%20EDVAC.pdf>

The von Neumann computer



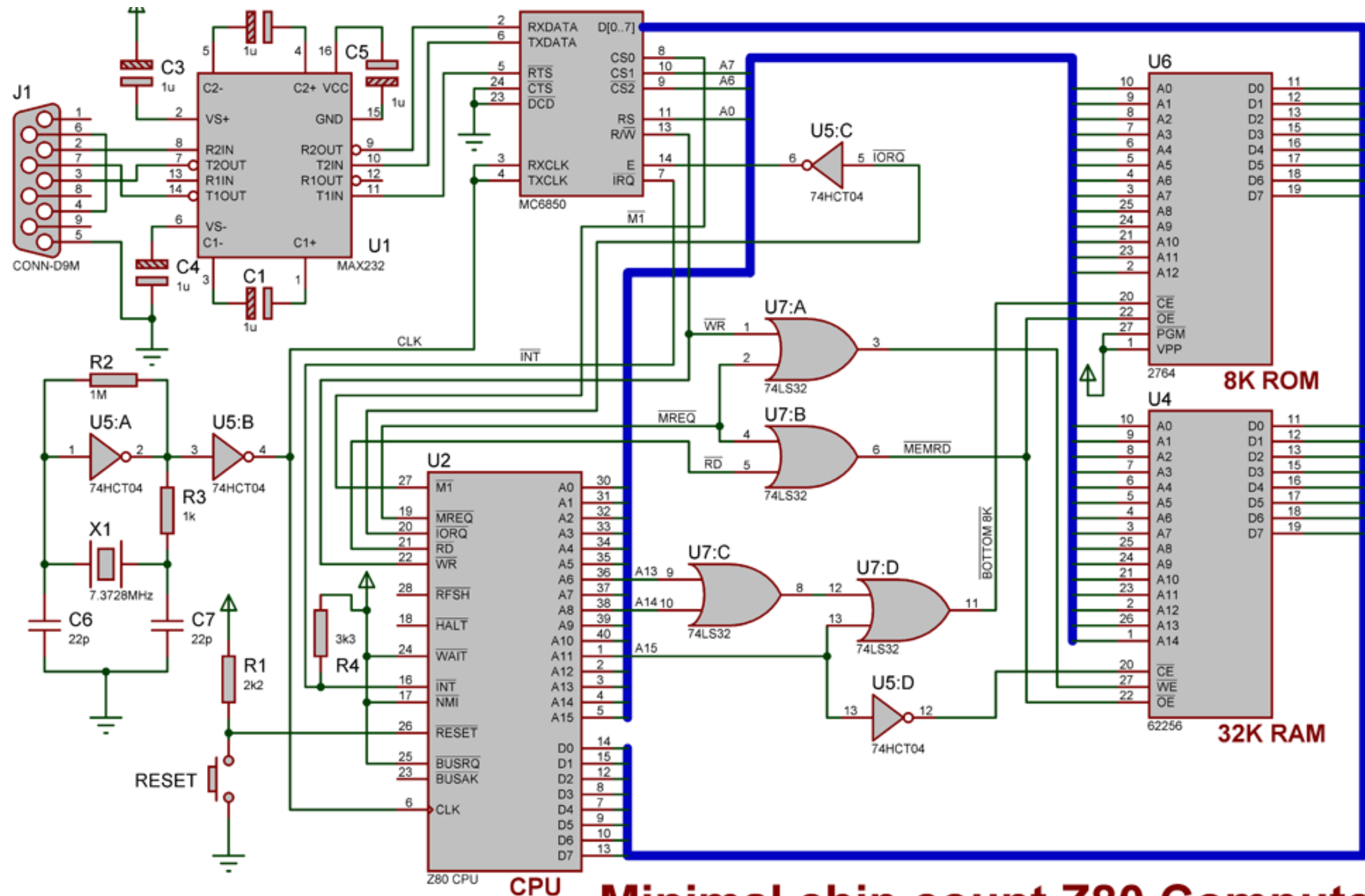
What is a bus actually?



(Go to next slide for bigger version)

Minimal chip count Z80 Computer
(32K RAM version) by Grant Searle 2007

<https://www.madexp.it/2018/02/26/z80-single-board-computer/>



CPU

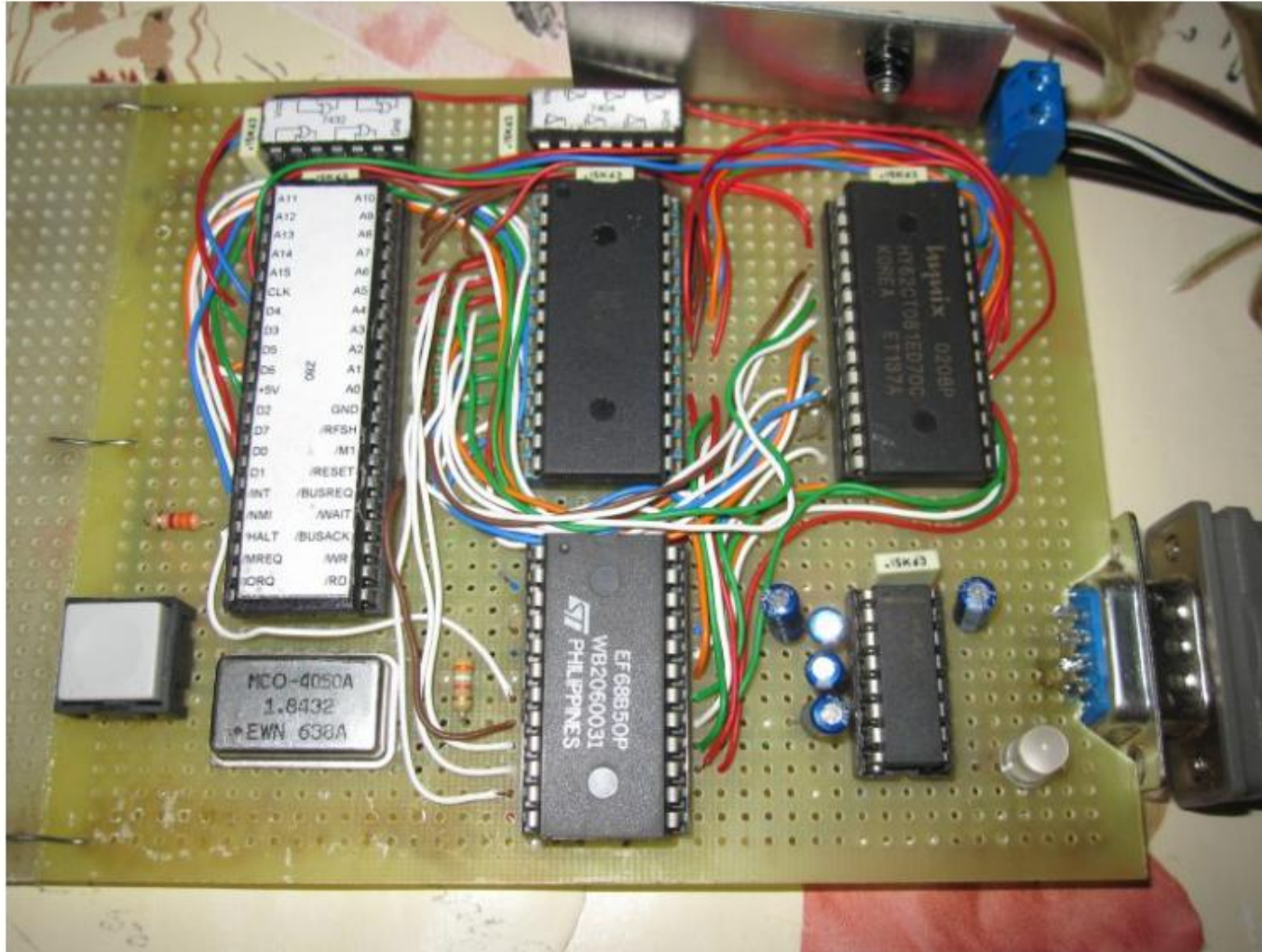
Minimal Chipset Z80 Computer

Note shared signals:

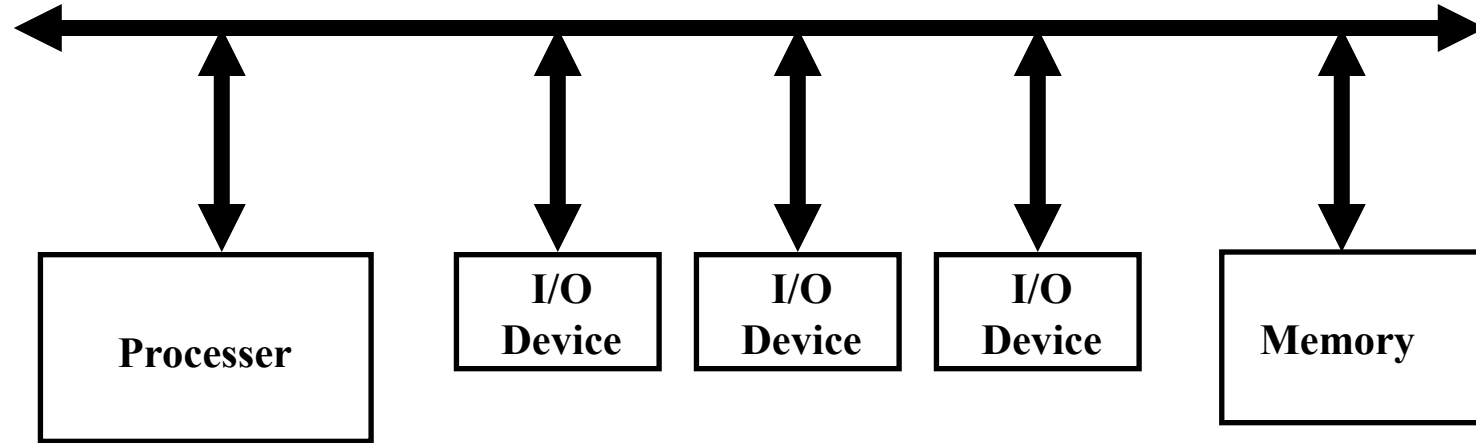
- Address
 - Even this simple system has a “memory map” – note use of A15 to select
- Data
- Some “extra stuff”
- Note U3 also on the bus!



Physical Implementation

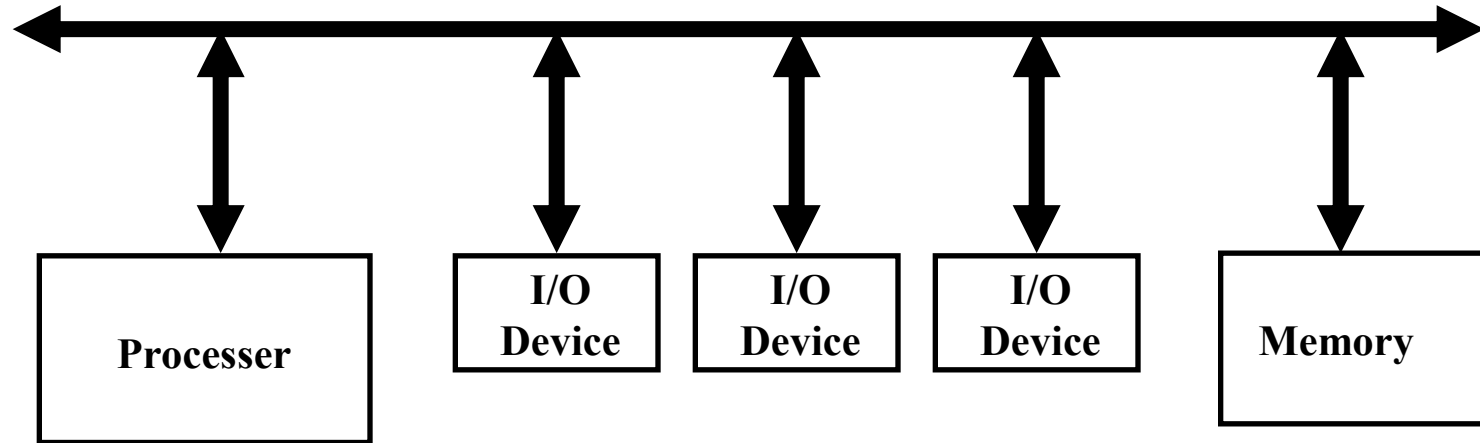


Advantages of Buses



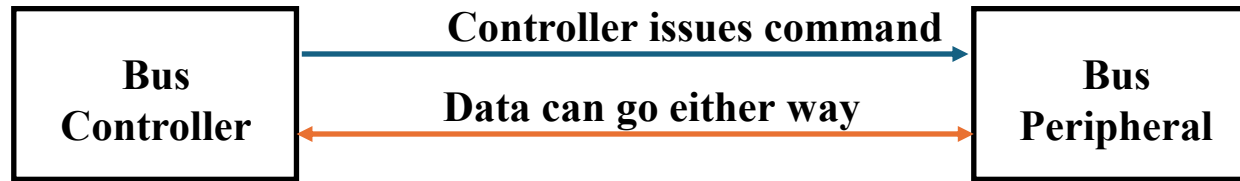
- **Versatility:**
 - New devices can be added easily
 - Peripherals can be moved between computer systems that use the same bus standard
- **Low Cost:**
 - A single set of wires is shared in multiple ways

Disadvantages of Buses



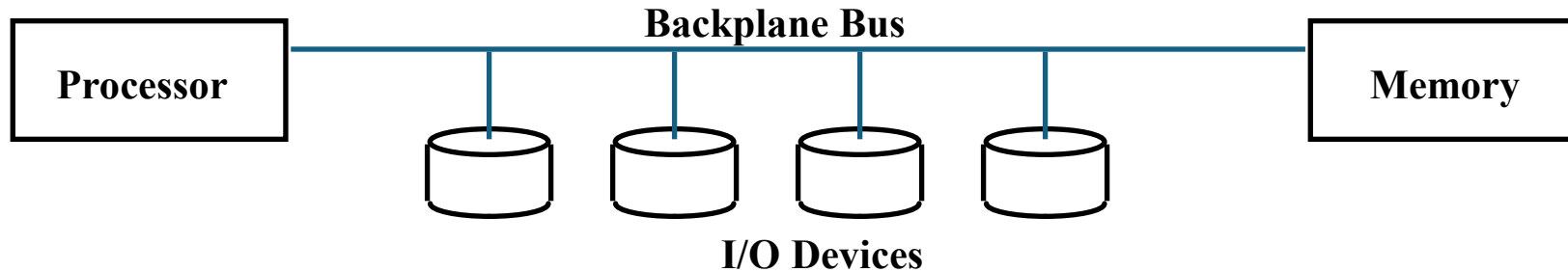
- It creates a communication bottleneck
 - The bandwidth of that bus can limit the maximum I/O throughput
- The maximum bus speed is largely limited by:
 - The **length** of the bus
 - The **number** of devices on the bus
 - The need to support a range of devices with:
 - Widely varying latencies
 - Widely varying data transfer rates

Bus Controller vs. Peripheral



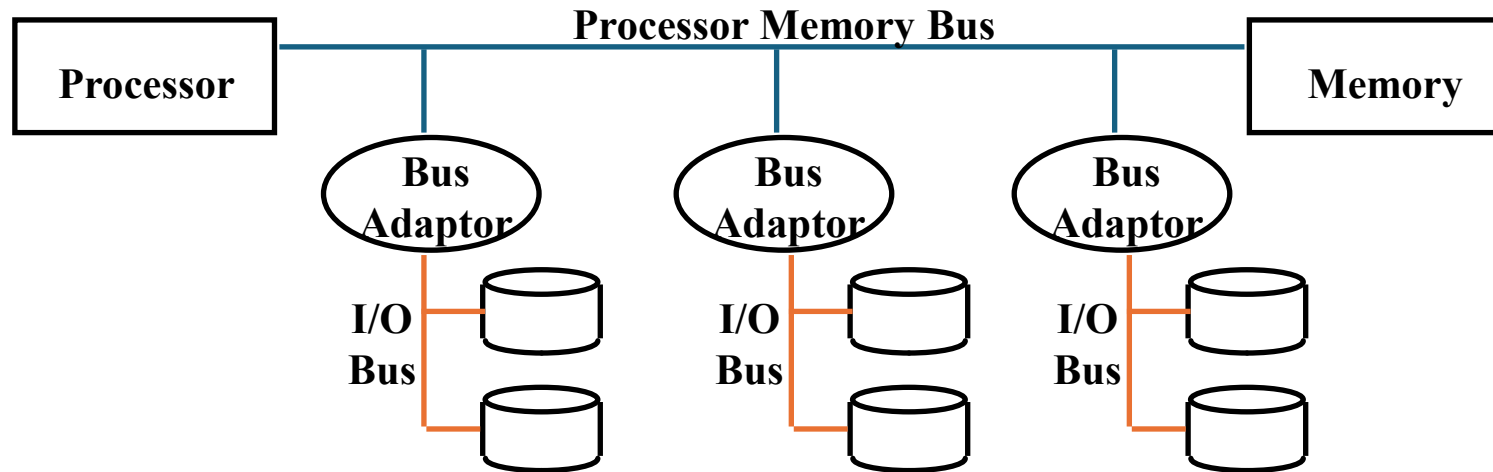
- A bus transaction includes two parts:
 - Issuing the command (and address) – request
 - Transferring the data – action
- Controller is the one who starts the bus transaction by:
 - issuing the command (and address)
- Peripheral is the one who responds to the address by:
 - Sending data to the controller if the controller ask for data
 - Receiving data from the controller if the controller wants to send data

Bus Organization: One Bus



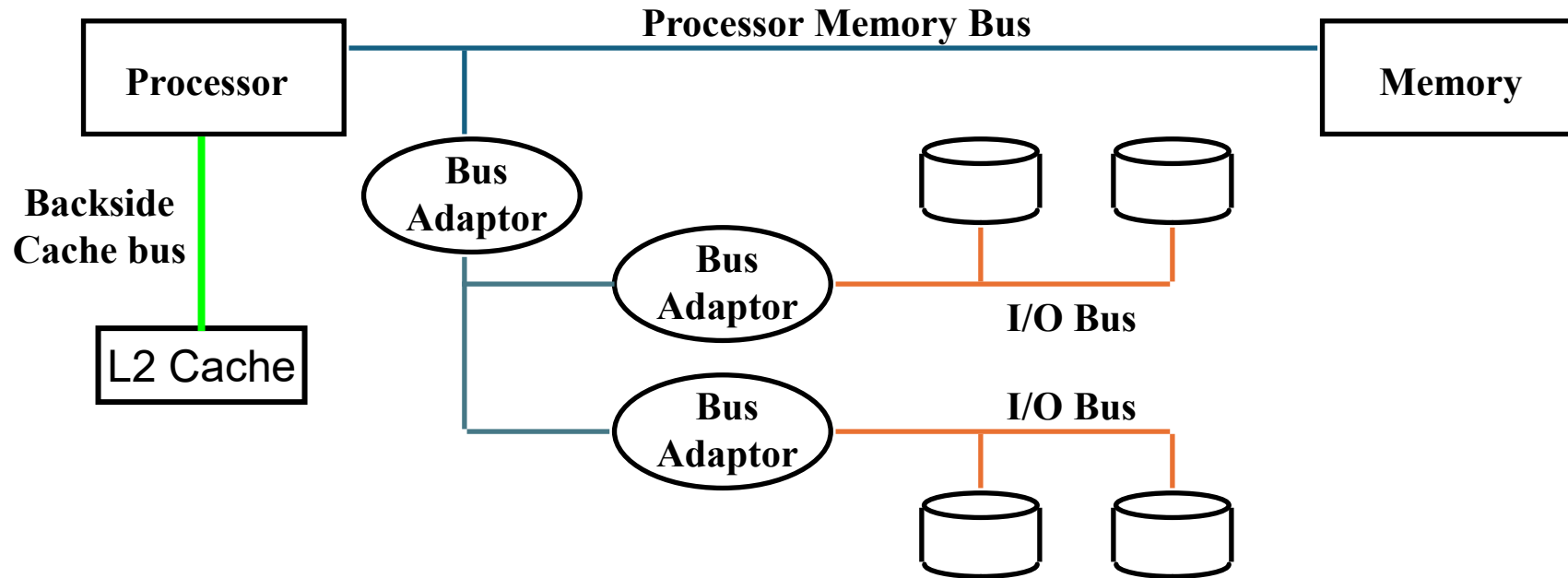
- A single bus (the backplane bus) is used for:
 - Processor to memory communication
 - Communication between I/O devices and memory
- Advantages: Simple and low cost
- Disadvantages: slow and the bus can become a major bottleneck
- Example: IBM PC - AT

Bus Organization: Two Buses



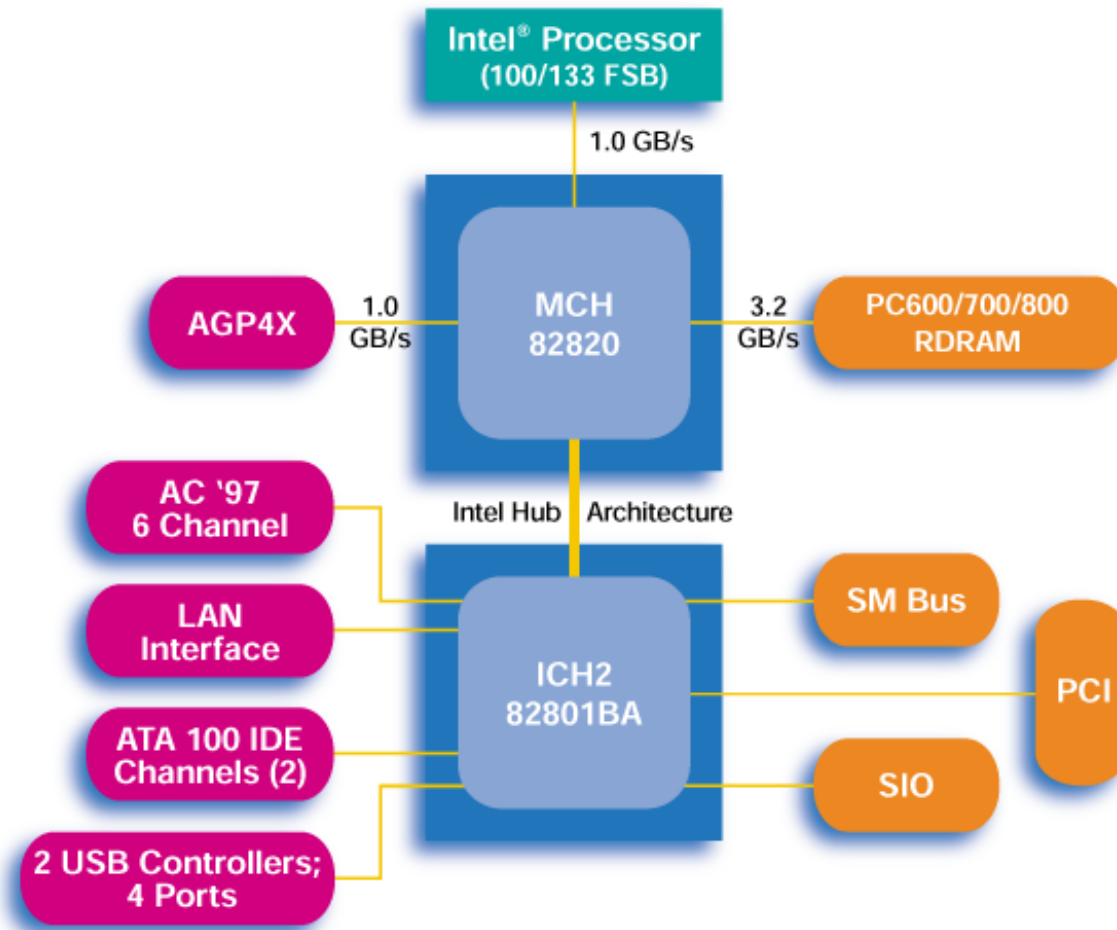
- I/O buses tap into the processor-memory bus via bus adaptors:
 - Processor-memory bus: mainly for processor-memory traffic
 - I/O buses: provide expansion slots for I/O devices
- Apple Macintosh-II
 - NuBus: Processor, memory, and a few selected I/O devices
 - SCCI Bus: the rest of the I/O devices

Bus Organization: Three Busses



- A small number of backplane buses tap into the processor-memory bus
 - Processor-memory bus is only used for processor-memory traffic
 - I/O buses are connected to the backplane bus
- Advantage: loading on the processor bus is greatly reduced

e.g., Old Pentium II/III Organization:

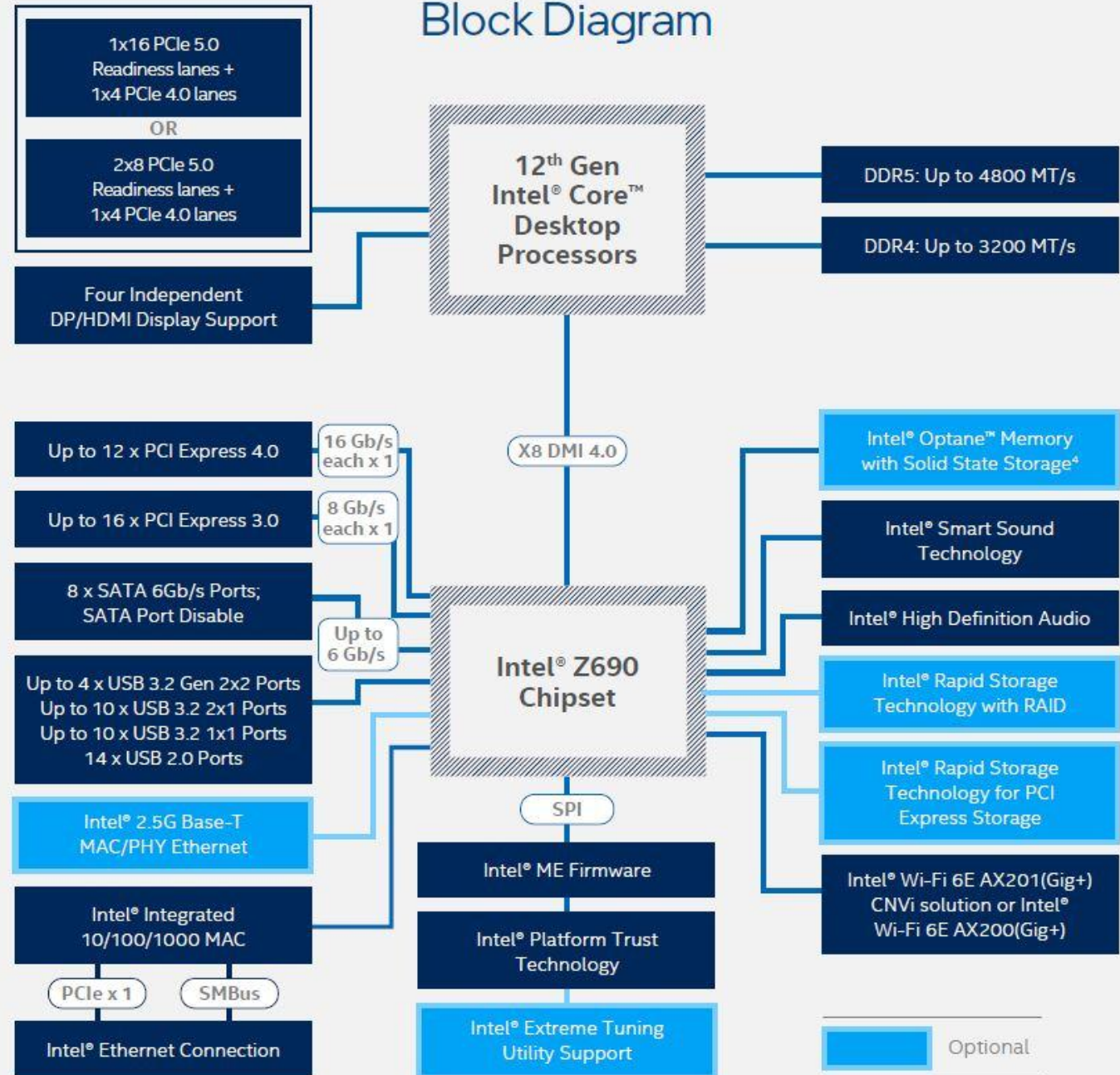


e.g., Newer Gen

- More functions now integrated into the main chip.
- External busses are always slower...

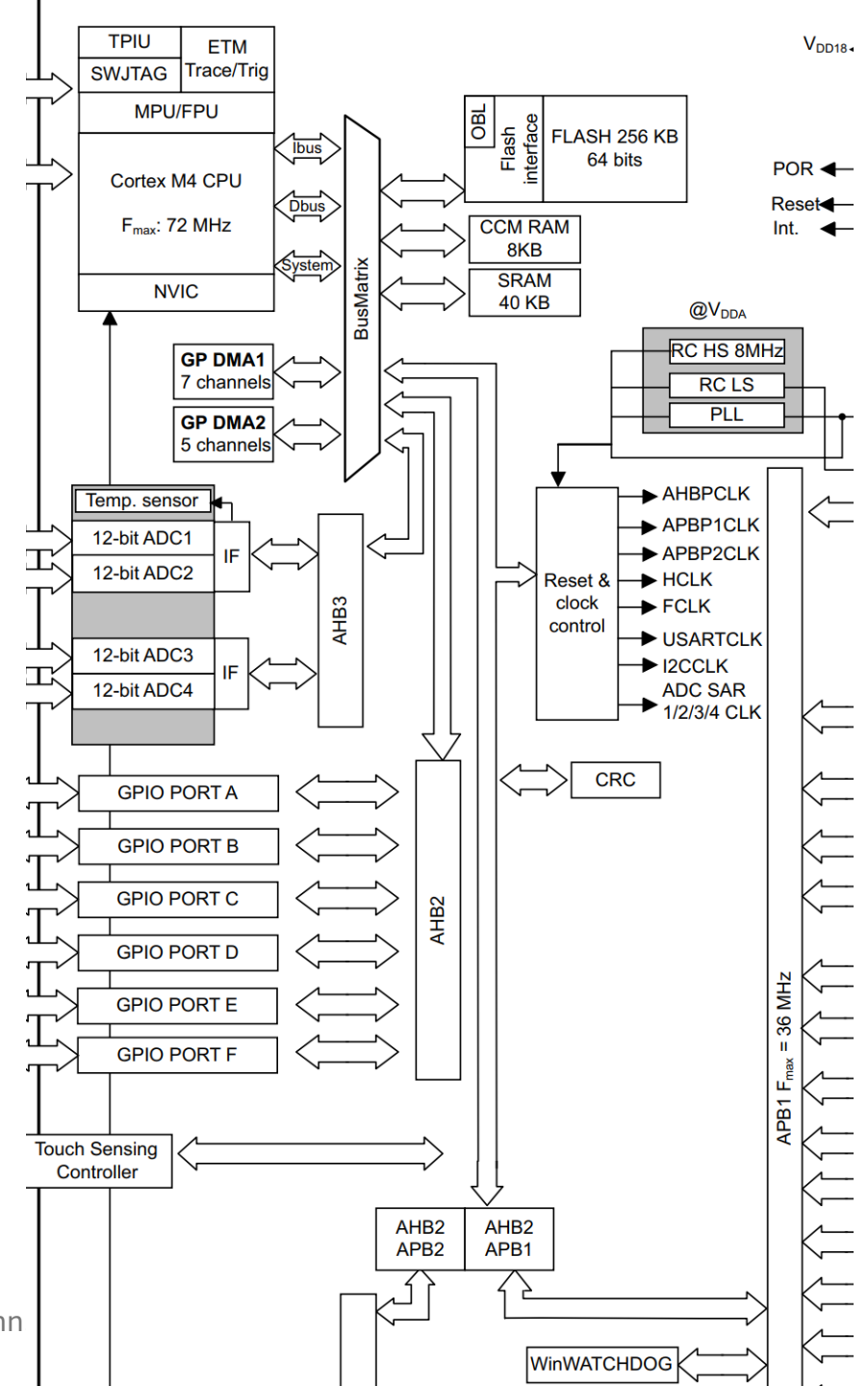
ECED

Intel® Z690 Chipset Block Diagram



Embedded Processor Busses

- Busses serve similar purposes even if not external.
- Standard bus interface makes it easy to add/remove blocks



Common *On-Chip* Bus Protocols

- **AHB** (**A**dvanced **H**igh-performance **B**us)
 - Large bus widths supported, advanced features like bursts
- **APB** (**A**dvanced **P**eripheral **B**us)
 - Similar to AHB but simpler signaling, designed for low-frequency peripherals
- **AXI** (**A**dvanced **eX**tensible **I**nterface)
 - Higher performance than AHB – allows more features like multiple “outstanding transfers”

Transfer Modes in Busses

- Naïve Bus Protocol Implementation (hope data is ready)



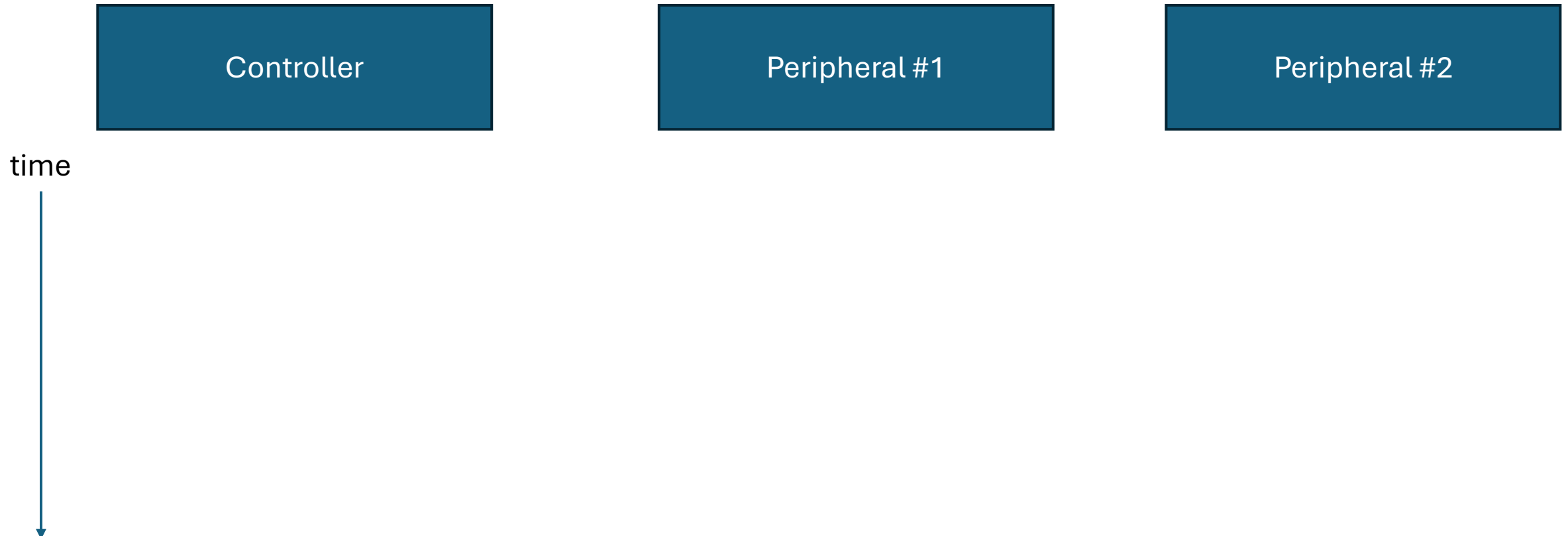
Transfer Modes in Busses

- Realistic Bus Protocol Implementation (acknowledge signal)

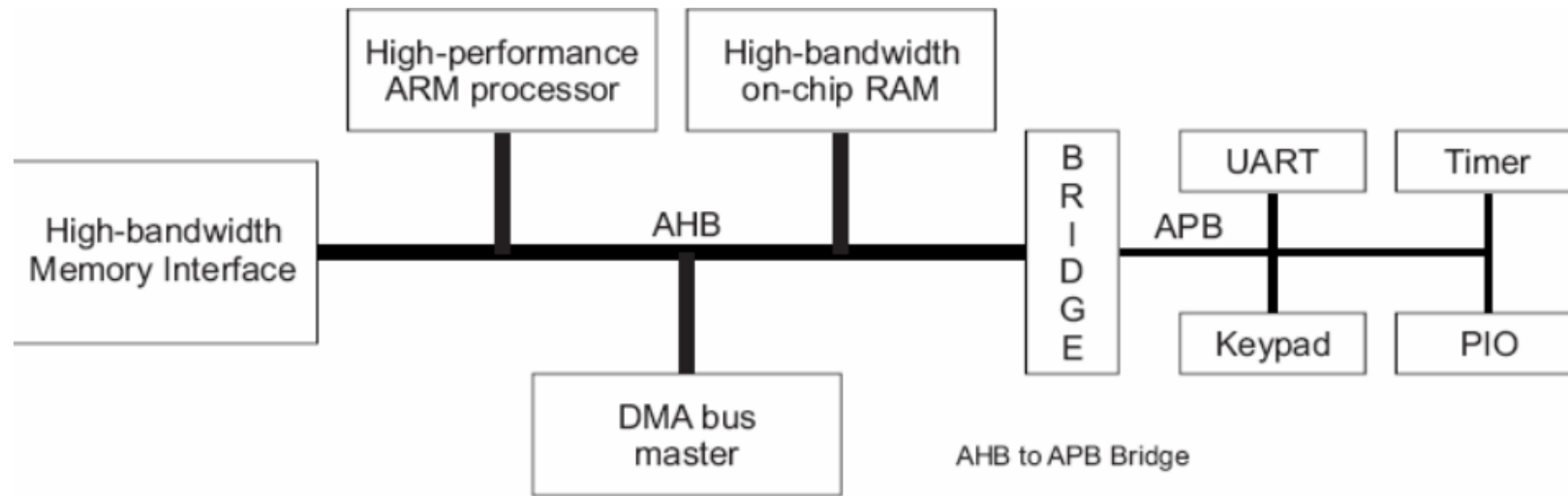


Transfer Modes in Busses

- Avoiding Stalls: Outstanding Transactions



AHB Signaling



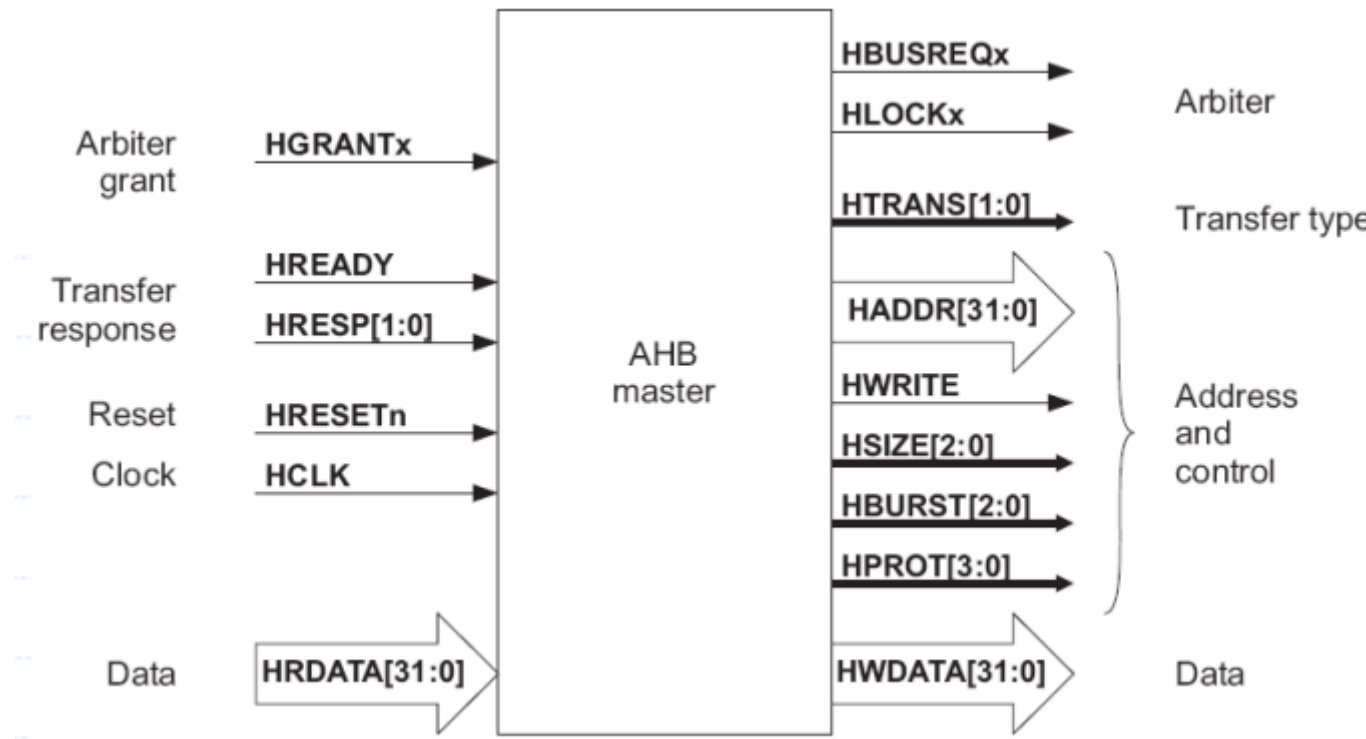
AMBA Advanced High-performance Bus (AHB)

- * High performance
- * Pipelined operation
- * Burst transfers
- * Multiple bus masters
- * Split transactions

AMBA Advanced Peripheral Bus (APB)

- * Low power
- * Latched address and control
- * Simple interface
- * Suitable for many peripherals

AHB Signals

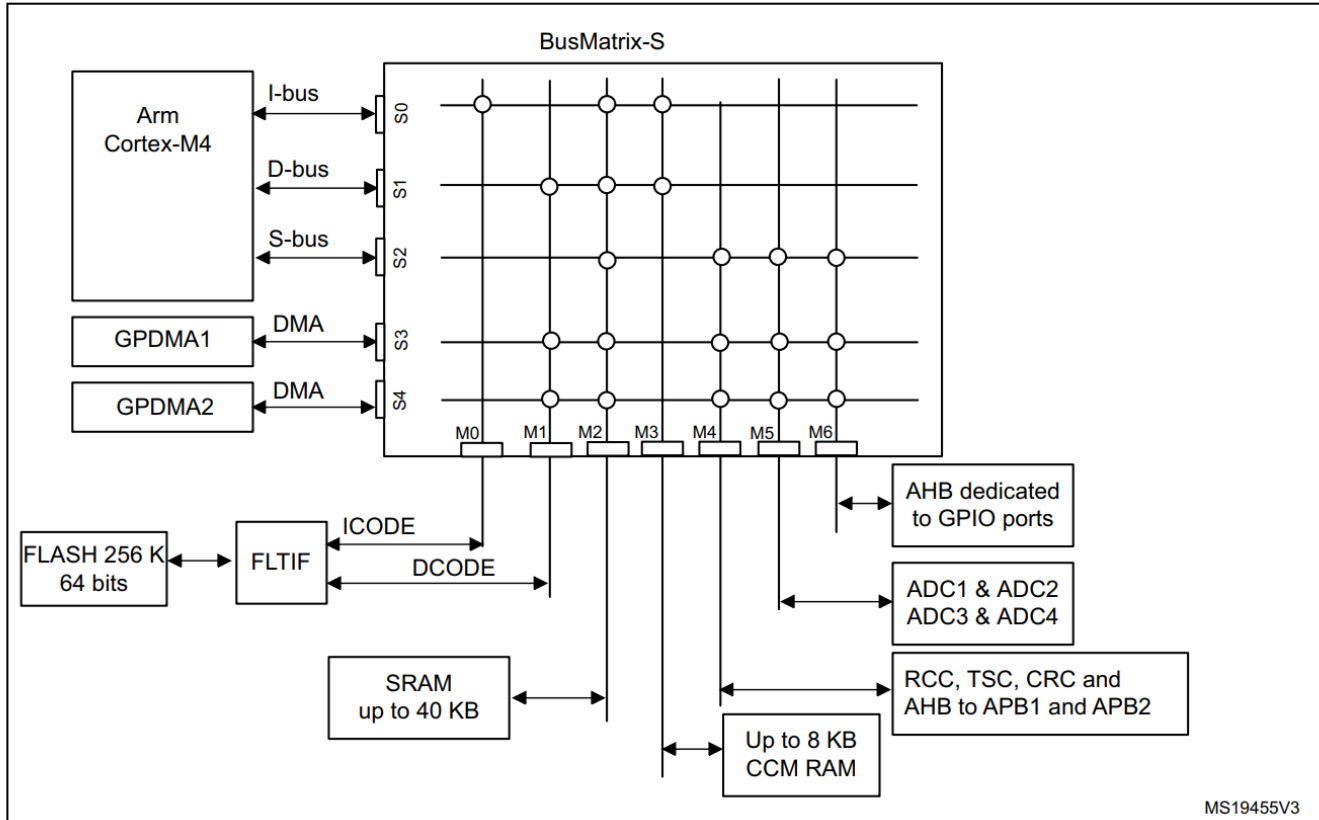


Typical Internal Bus Signaling

- Not “pin limited”
 - Note separate input & output data lines
 - Common to have many control signals

e.g., STM32F3 Bus Matrix

Figure 1. STM32F303xB/C and STM32F358xC system architecture



3.1.1 S0: I-bus

This bus connects the instruction bus of the Cortex[®]-M4 core to the BusMatrix. This bus is used by the core to fetch instructions. The targets of this bus are the internal Flash memory, the SRAM and the CCM SRAM.

3.1.2 S1: D-bus

This bus connects the DCode bus (literal load and debug access) of the Cortex[®]-M4 core to the BusMatrix. The targets of this bus are the internal Flash memory, the SRAM and the CCM SRAM.

3.1.3 S2: S-bus

This bus connects the system bus of the Cortex[®]-M4 core to the BusMatrix. This bus is used to access data located in the peripheral or SRAM area. The targets of this bus are the SRAM, the AHB to APB1/APB2 bridges, the AHB IO port and the ADC.

3.1.4 S3, S4: DMA-bus

This bus connects the AHB master interface of the DMA to the BusMatrix, which manages the access of different Masters to flash, SRAM, and peripherals.

Bus Arbitration?

- A bus may have more than one controller:
 - Multi-core processors
 - Systems with e.g., a high-performance & low-performance processor
 - Special blocks such as DMA
- Bus arbitration determines *who* owns the bus, various algorithms used, but e.g., STM32F3:

3.1.5 BusMatrix

The BusMatrix manages the access arbitration between Masters. The arbitration uses a Round Robin algorithm. The BusMatrix is composed of five masters (CPU AHB, System bus, DCode bus, ICode bus, DMA1/2 bus) and seven slaves (FLITF, SRAM, AHB2GPIO and AHB2APB1/2 bridges, and ADC).

Round Robin

DMA2

Core: I-Bus

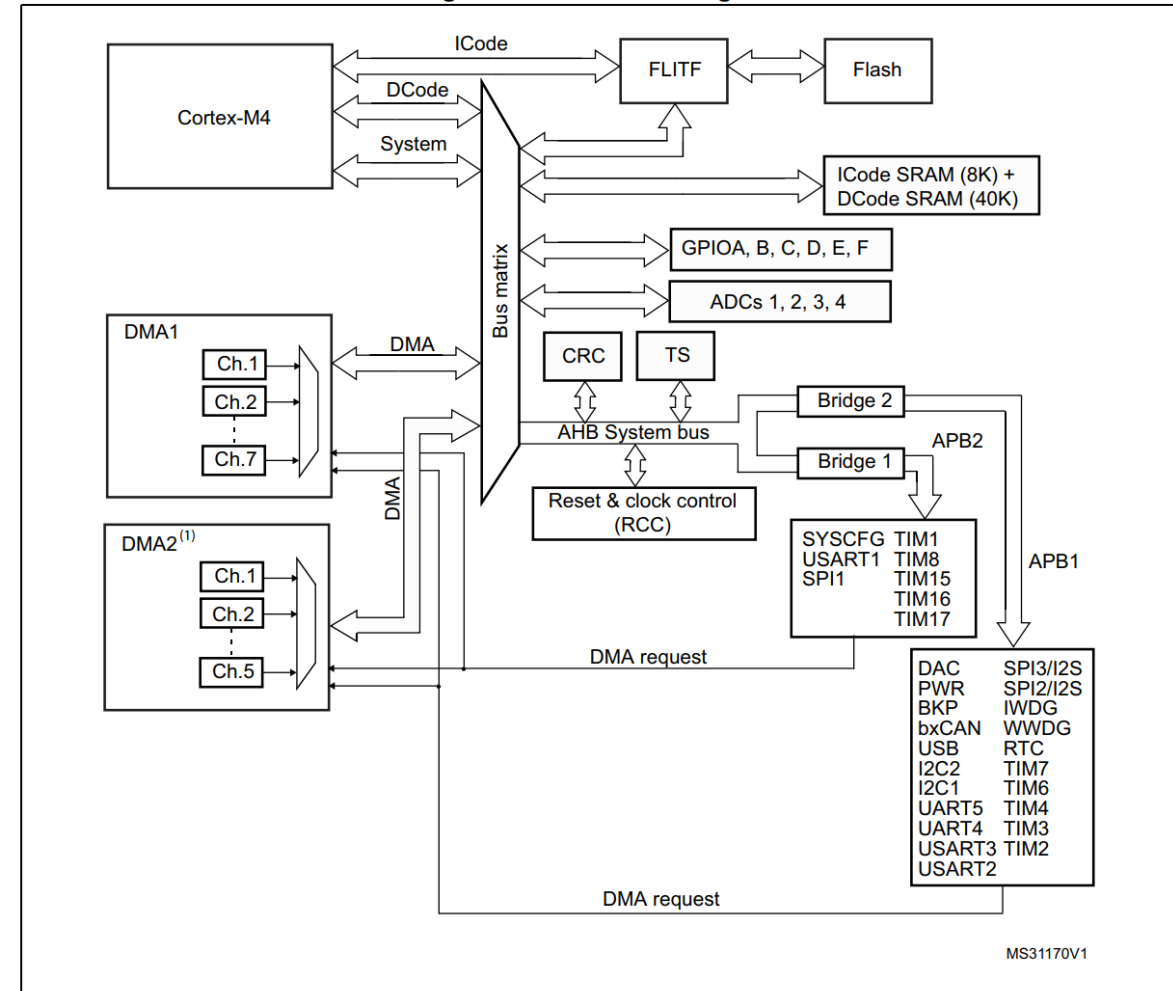
DMA1

Core: D-Bus

DMA

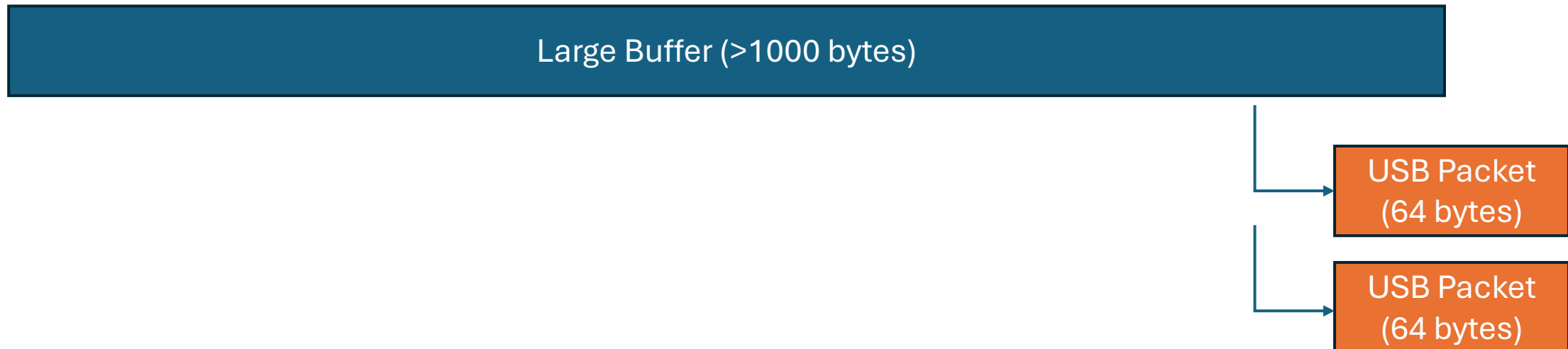
- DMA is another bus controller which can read/write from Memory & Peripherals.
- Can be triggered by events such as:
 - ADC complete
 - Packet received (USB, Ethernet, etc)

Figure 49. DMA block diagram



DMA Keeps the CPU Free

- Examples of common DMA Usage:
 - Copy large blocks of data
 - Automatically receive incoming data to a buffer
 - Take a transmit buffer, read a new “chunk” of data when the peripheral is ready



DMA Usage

- DMA is one of the most powerful features of microcontrollers you can use with high I/O requirements.
- Many programs are only possible with DMA, for example:
 - USB Microphone would have:
 - DMA from ADC → Memory
 - DMA from Memory → USB
 - Microcontroller does “housekeeping” keeping signals alive
- Normally required for sampling & high-throughput communications

e.g., DMA Demo

```
11
12 uint32_t * src = (uint32_t *)0x20001000;
13 uint32_t * dst = (uint32_t *)0x20002000;
14
15 static void dma1_ch1_mem2mem_init(void) {
16     RCC->AHBENR |= RCC_AHBENR_DMA1EN;
17
18     DMA1_Channel1->CCR &= ~DMA_CCR_EN;
19
20     // Clear all ch1 flags
21     DMA1->IFCR = DMA_IFCR_CGIF1 | DMA_IFCR_CTCIF1 | DMA_IFCR_CHTIF1 | DMA_IFCR_CTEIF1;
22
23     // MEM2MEM + increment both + 32-bit width
24     // IMPORTANT: leave DIR=0 (P->M). In MEM2MEM, CPAR is treated as "source".
25     DMA1_Channel1->CCR =
26         DMA_CCR_MEM2MEM |
27         DMA_CCR_MINC |           // increment destination (CMAR)
28         DMA_CCR_PINC |           // increment source (CPAR)
29         DMA_CCR_MSIZE_1 |        // 32-bit
30         DMA_CCR_PSIZE_1 |        // 32-bit
31         DMA_CCR_PL_1;            // high priority (optional)
32 }
33
34 static void start_dma_memcpy_u32(const uint32_t *from, uint32_t *to, uint16_t nwords) {
35     DMA1_Channel1->CCR &= ~DMA_CCR_EN;
36
37     DMA1->IFCR = DMA_IFCR_CGIF1 | DMA_IFCR_CTCIF1 | DMA_IFCR_CHTIF1 | DMA_IFCR_CTEIF1;
38
39     // In MEM2MEM, CPAR = source, CMAR = destination
40     DMA1_Channel1->CPAR = (uint32_t)from;
41     DMA1_Channel1->CMAR = (uint32_t)to;
42     DMA1_Channel1->CNDTR = nwords;
43
44     __DSB();
45     DMA1_Channel1->CCR |= DMA_CCR_EN;
46 }
```

Sidenote: Data Synchronization Barrier (DSB)

- Note the Data Synchronization Barrier (DSB)
 - Technically not needed as peripheral memory should not be reordered
 - But if you were writing something to e.g., memory being read from before enabling may still want that (ensures memory consistency)

DMA Usage: High-Level

- Set source & destination addresses
- Set if either address should automatically increment (normally **not** incrementing for peripherals, but **do** increment for memory)
- Set word size (e.g., normally 32 bits)
- Set number of **words** to transfer
 - Watch out for Bytes/Words confusion!
- Enable & let it go!

STM32F3 DMA Hints

13.6	DMA registers	280
13.6.1	DMA interrupt status register (DMA_ISR)	280
13.6.2	DMA interrupt flag clear register (DMA_IFCR)	283
13.6.3	DMA channel x configuration register (DMA_CCRx)	284
13.6.4	DMA channel x number of data to transfer register (DMA_CNDTRx) .	287
13.6.5	DMA channel x peripheral address register (DMA_CPARx)	287
13.6.6	DMA channel x memory address register (DMA_CMARx)	288
13.6.7	DMA register map	288

- CPAR / CMAR can be source/dest depending on “DIR” bit in CCR
- Watch out that not all memory addresses are accessible (see next topic on TCM/CCM)

STM32F3: DMA Data Width & Alignments

- Many DMA problems (on all devices) are related to data widths.
- If not transferring the same data width, need to be very careful about order/alignment.
- E.g., Table at right is from ST32F303 Ref Manual

Table 79. Programmable data width and endian behavior (when PINC = MINC = 1)

Source port width (MSIZE) if DIR = 1, else PSIZE)	Destination port width (PSIZE) if DIR = 1, else MSIZE)	Number of data items to transfer (NDT)	Source content: address / data (DMA_CMARx if DIR = 1, else DMA_CPARx)	DMA transfers	Destination content: address / data (DMA_CPARx if DIR = 1, else DMA_CMARx)
8	8	4	@0x0 / B0 @0x1 / B1 @0x2 / B2 @0x3 / B3	1: read B0[7:0] @0x0 then write B0[7:0] @0x0 2: read B1[7:0] @0x1 then write B1[7:0] @0x1 3: read B2[7:0] @0x2 then write B2[7:0] @0x2 4: read B3[7:0] @0x3 then write B3[7:0] @0x3	@0x0 / B0 @0x1 / B1 @0x2 / B2 @0x3 / B3
8	16	4	@0x0 / B0 @0x1 / B1 @0x2 / B2 @0x3 / B3	1: read B0[7:0] @0x0 then write 00B0[15:0] @0x0 2: read B1[7:0] @0x1 then write 00B1[15:0] @0x2 3: read B2[7:0] @0x2 then write 00B2[15:0] @0x4 4: read B3[7:0] @0x3 then write 00B3[15:0] @0x6	@0x0 / 00B0 @0x2 / 00B1 @0x4 / 00B2 @0x6 / 00B3
8	32	4	@0x0 / B0 @0x1 / B1 @0x2 / B2 @0x3 / B3	1: read B0[7:0] @0x0 then write 000000B0[31:0] @0x0 2: read B1[7:0] @0x1 then write 000000B1[31:0] @0x4 3: read B2[7:0] @0x2 then write 000000B2[31:0] @0x8 4: read B3[7:0] @0x3 then write 000000B3[31:0] @0xC	@0x0 / 000000B0 @0x4 / 000000B1 @0x8 / 000000B2 @0xC / 000000B3
16	8	4	@0x0 / B1B0 @0x2 / B3B2 @0x4 / B5B4 @0x6 / B7B6	1: read B1B0[15:0] @0x0 then write B0[7:0] @0x0 2: read B3B2[15:0] @0x2 then write B2[7:0] @0x1 3: read B5B4[15:0] @0x4 then write B4[7:0] @0x2 4: read B7B6[15:0] @0x6 then write B6[7:0] @0x3	@0x0 / B0 @0x1 / B2 @0x2 / B4 @0x3 / B6
16	16	4	@0x0 / B1B0 @0x2 / B3B2 @0x4 / B5B4 @0x6 / B7B6	1: read B1B0[15:0] @0x0 then write B1B0[15:0] @0x0 2: read B3B2[15:0] @0x2 then write B3B2[15:0] @0x2 3: read B5B4[15:0] @0x4 then write B5B4[15:0] @0x4 4: read B7B6[15:0] @0x6 then write B7B6[15:0] @0x6	@0x0 / B1B0 @0x2 / B3B2 @0x4 / B5B4 @0x6 / B7B6
16	32	4	@0x0 / B1B0 @0x2 / B3B2 @0x4 / B5B4 @0x6 / B7B6	1: read B1B0[15:0] @0x0 then write 0000B1B0[31:0] @0x0 2: read B3B2[15:0] @0x2 then write 0000B3B2[31:0] @0x4 3: read B5B4[15:0] @0x4 then write 0000B5B4[31:0] @0x8 4: read B7B6[15:0] @0x6 then write 0000B7B6[31:0] @0xC	@0x0 / 0000B1B0 @0x4 / 0000B3B2 @0x8 / 0000B5B4 @0xC / 0000B7B6
32	8	4	@0x0 / B3B2B1B0 @0x4 / B7B6B5B4 @0x8 / BBBAB9B8 @0xC / BFBEBDBC	1: read B3B2B1B0[31:0] @0x0 then write B0[7:0] @0x0 2: read B7B6B5B4[31:0] @0x4 then write B4[7:0] @0x1 3: read BBBAB9B8[31:0] @0x8 then write B8[7:0] @0x2 4: read BFBEBDBC[31:0] @0xC then write B0[7:0] @0x3	@0x0 / B0 @0x1 / B4 @0x2 / B8 @0x3 / BC
32	16	4	@0x0 / B3B2B1B0 @0x4 / B7B6B5B4 @0x8 / BBBAB9B8 @0xC / BFBEBDBC	1: read B3B2B1B0[31:0] @0x0 then write B1B0[15:0] @0x0 2: read B7B6B5B4[31:0] @0x4 then write B5B4[15:0] @0x2 3: read BBBAB9B8[31:0] @0x8 then write B9B8[15:0] @0x4 4: read BFBEBDBC[31:0] @0xC then write BDBC[15:0] @0x6	@0x0 / B1B0 @0x2 / B5B4 @0x4 / B9B8 @0x6 / BDBC
32	32	4	@0x0 / B3B2B1B0 @0x4 / B7B6B5B4 @0x8 / BBBAB9B8 @0xC / BFBEBDBC	1: read B3B2B1B0[31:0] @0x0 then write B3B2B1B0[31:0] @0x0 2: read B7B6B5B4[31:0] @0x4 then write B7B6B5B4[31:0] @0x4 3: read BBBAB9B8[31:0] @0x8 then write BBBAB9B8[31:0] @0x8 4: read BFBEBDBC[31:0] @0xC then write BFBEBDBC[31:0] @0xC	@0x0 / B3B2B1B0 @0x4 / B7B6B5B4 @0x8 / BBBAB9B8 @0xC / BFBEBDBC

DMA for Data Packing

- Sometimes you use this on purpose:
 - 8-bit or 16-bit data coming in from a peripheral
 - You want this packed into 32-bit array for memory efficiency
 - DMA can do that automatically!

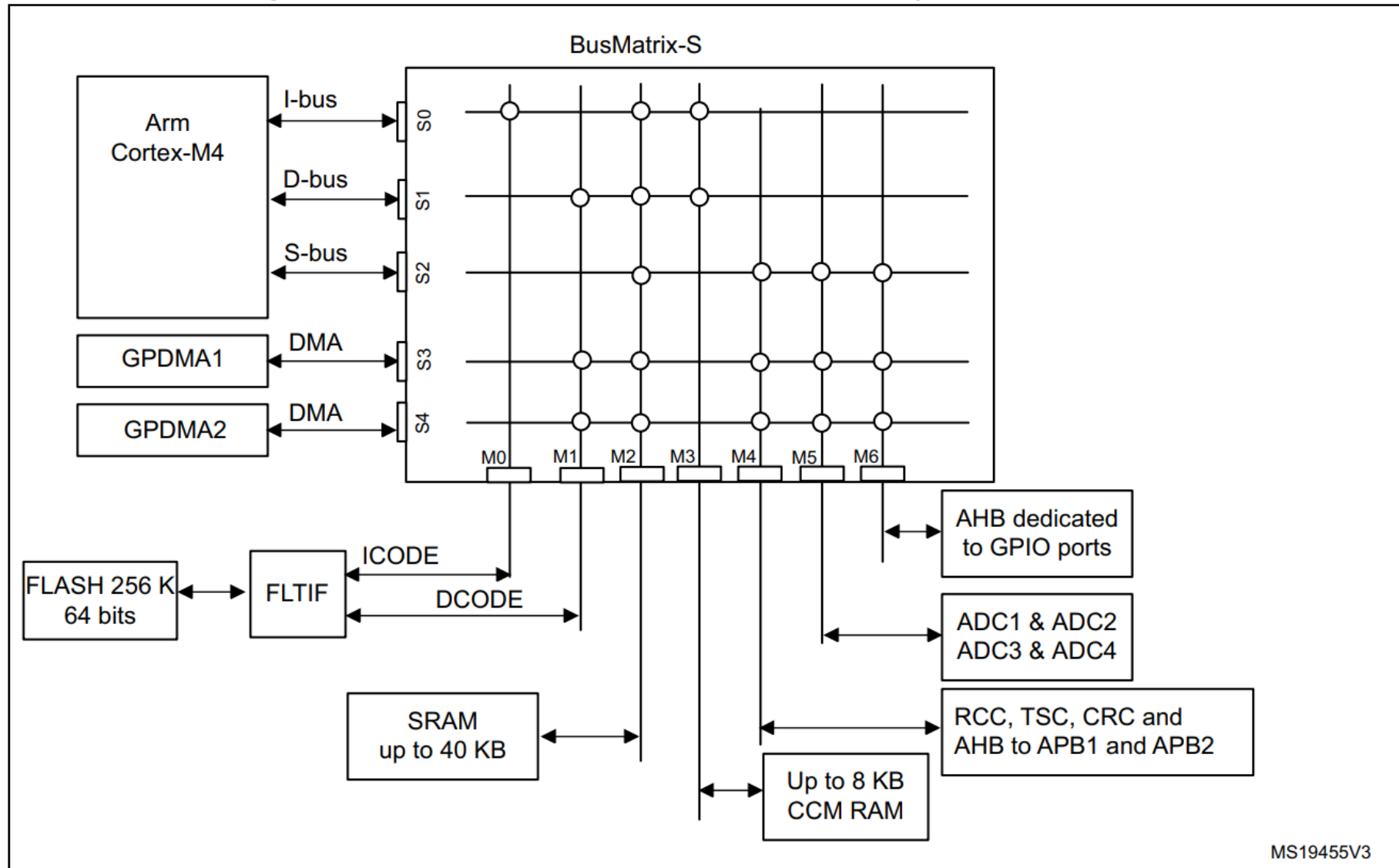
Core Coupled Memory (CCM)

Closely Coupled Memory (CCM)

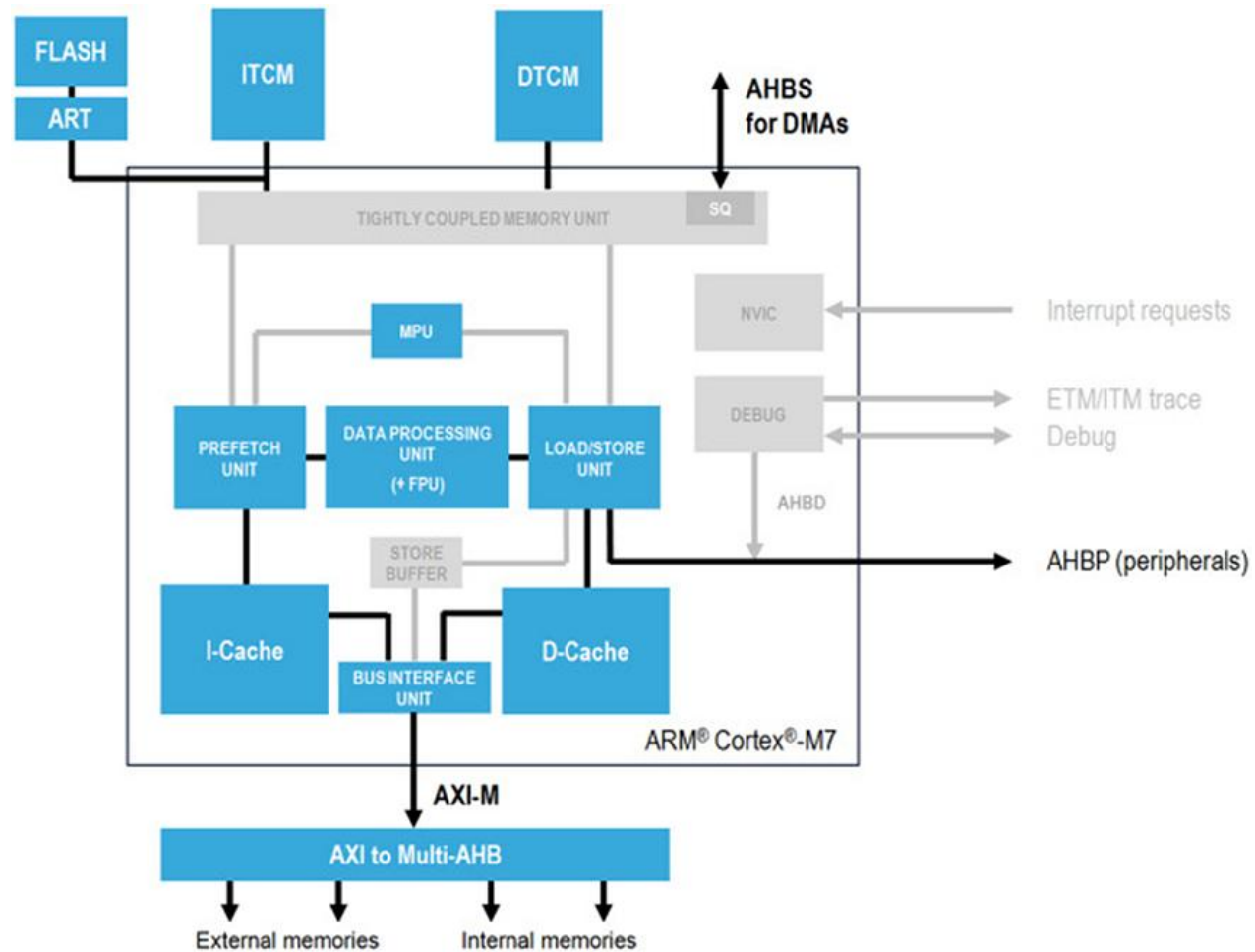
Tightly Coupled Memory (TCM)

- All of those have the same idea – keep some special RAM that is even faster than system memory.
 - Different than a cache as this is separate memory, not mirroring main memory.
 - TCM is present in higher-performance cores, may bypass matrix entirely.
 - Arm CCM uses matrix, but is not accessible to DMA as only for CPU usage.

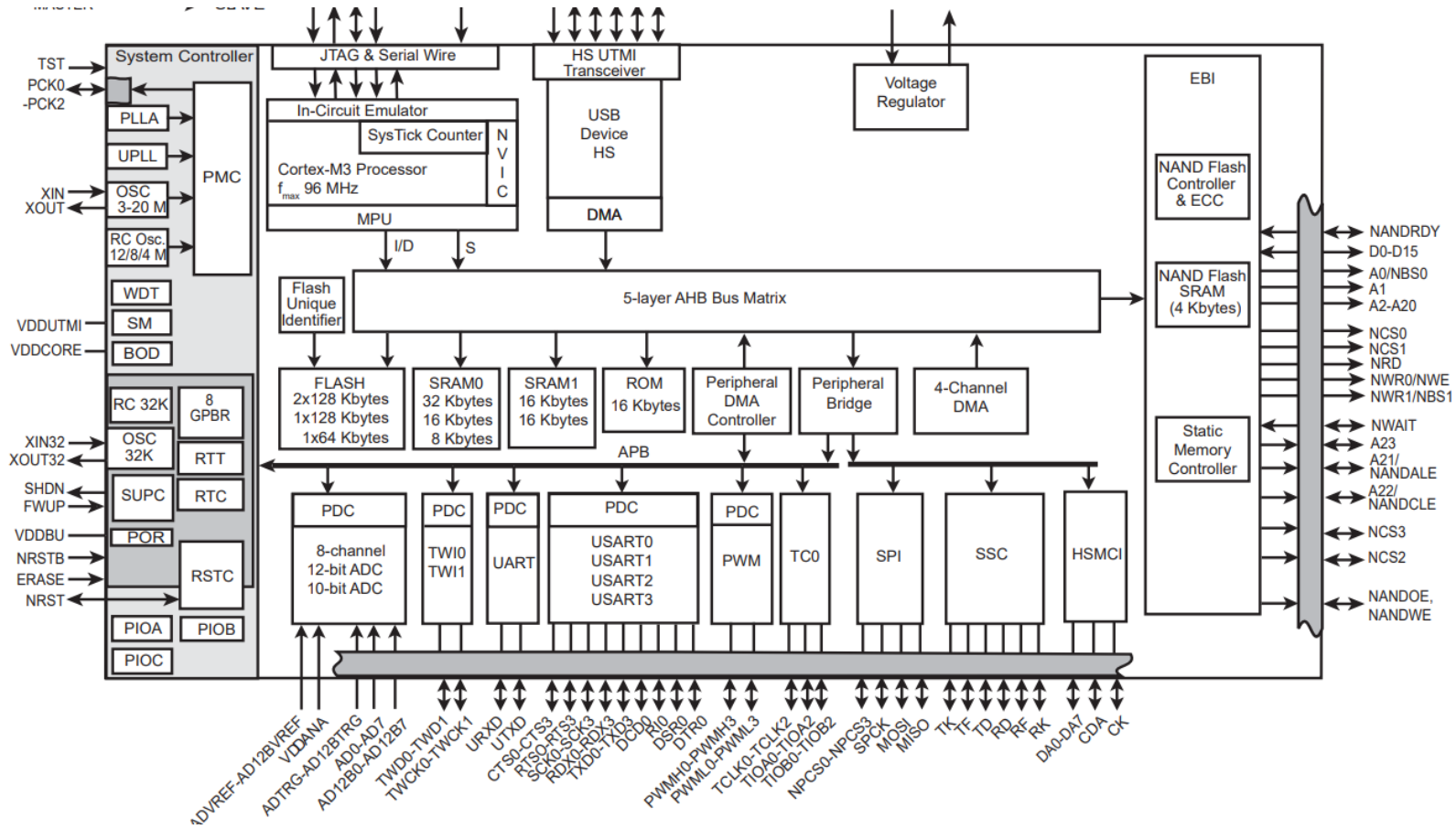
Figure 1. STM32F303xB/C and STM32F358xC system architecture



e.g., Cortex-M7 TCM Blocks



Simple External Busses: SRAM



- https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-6430-32-bit-Cortex-M3-Microcontroller-SAM3U4-SAM3U2-SAM3U1_Datasheet.pdf

Synchronous Bus

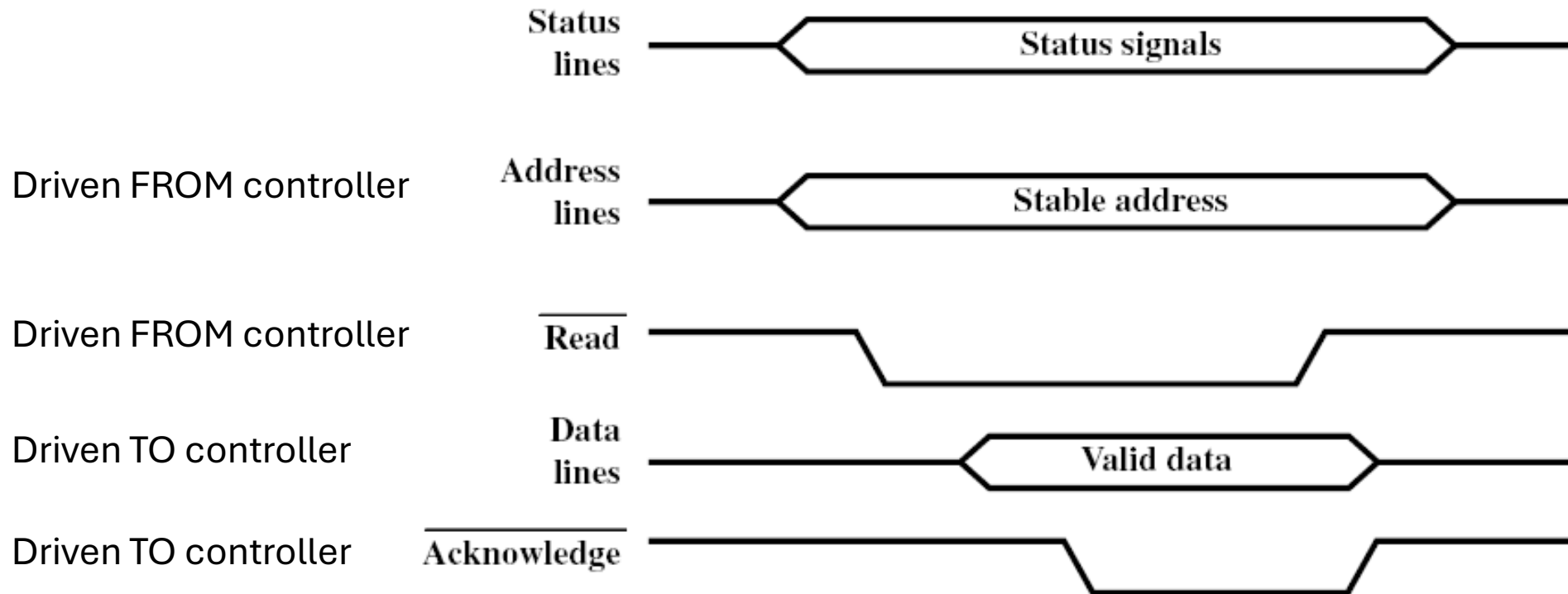
- Synchronous buses will have a clock
 - All data & control signals only read on the clock edge
 - Normally strict timing requirements
-
- Many internal busses are synchronous (e.g., AHB)
 - DDR Memory is an external example of synchronous bus

Asynchronous Bus

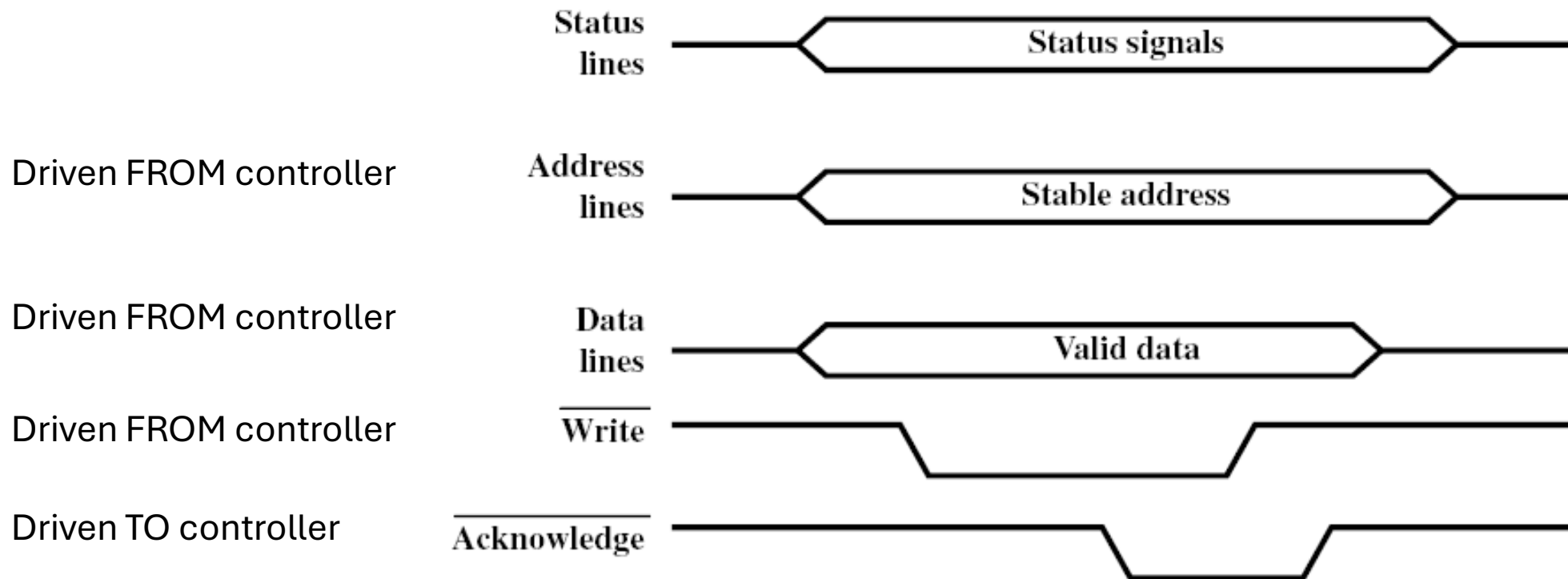
- Control signals define when data is ready or available
- Less strict timing requirements
- “Normal” (older) SRAM chips, many “slower” external memories are asynchronous



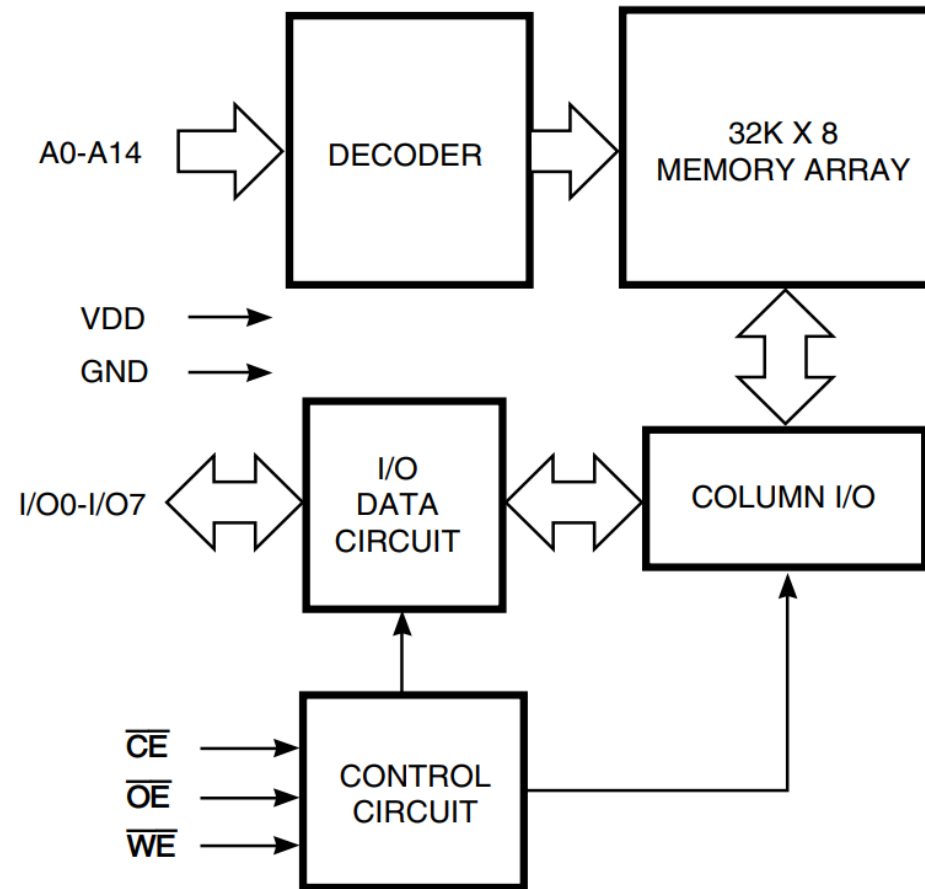
Async Read Example



Async Write Example



SRAM Configuration



e.g., Typical SRAM Signals

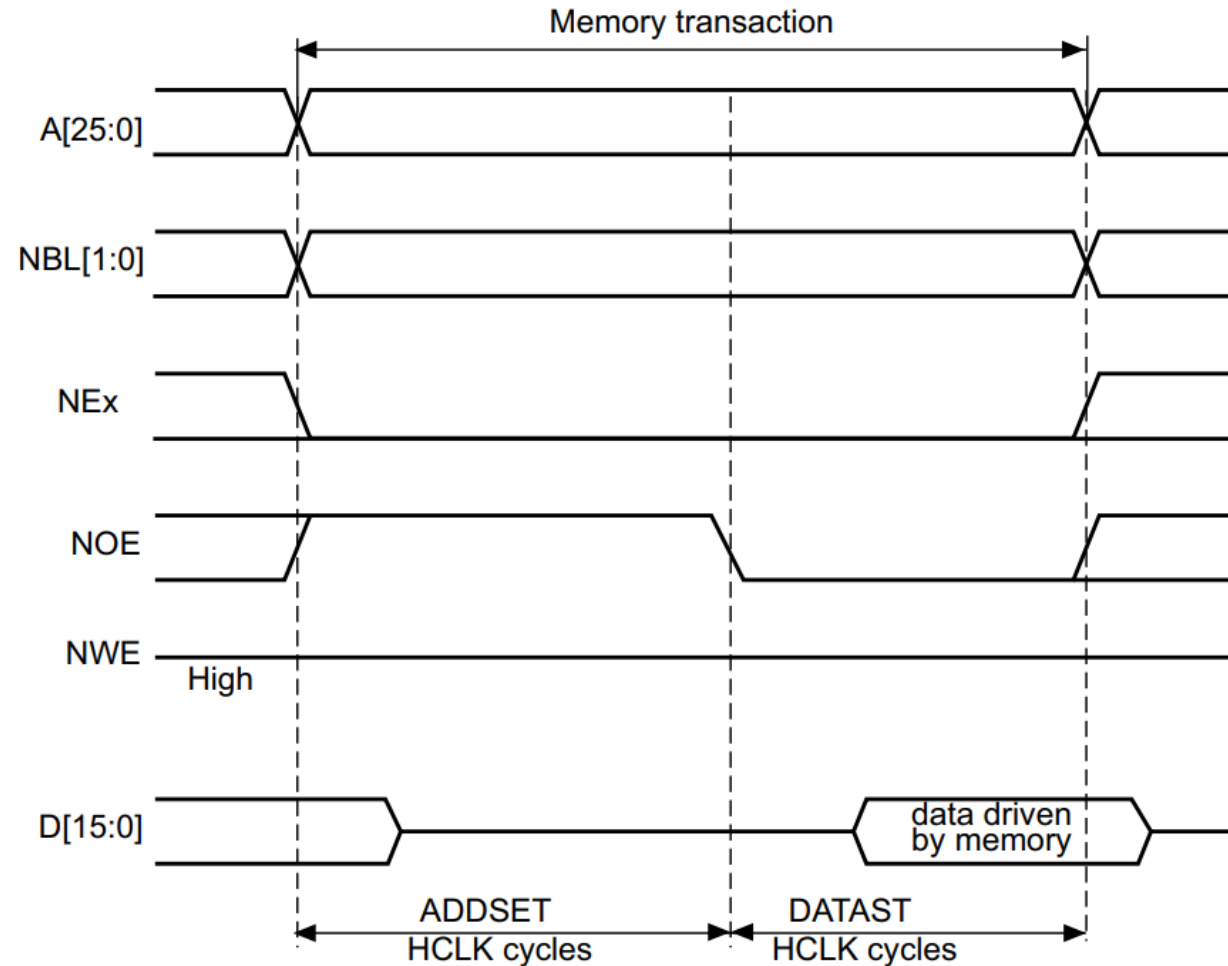
The maximum capacity is 512 Mbits.

PSRAM/SRAM, non-multiplexed I/Os

Table 41. Non-multiplexed I/Os PPSRAM/SRAM

FMC signal name	I/O	Function
CLK	O	Clock (only for PPSRAM synchronous access)
A[25:0]	O	Address bus
D[15:0]	I/O	Data bidirectional bus
NE[x]	O	Chip select, x = 1..4 (called NCE by PPSRAM (Cellular RAM i.e. CRAM))
NOE	O	Output enable
NWE	O	Write enable
NL(= NADV)	O	Address valid only for PPSRAM input (memory signal name: NADV)
NWAIT	I	PPSRAM wait input signal to the FMC
NBL[1:0]	O	Byte lane output. Byte 0 and byte 1 control (upper and lower byte enable)

SRAM Driven from STM32F3



NOTE: No
“acknowledge” signal!

This requires setting
up the timing, and the
SRAM responds on
definite timing.

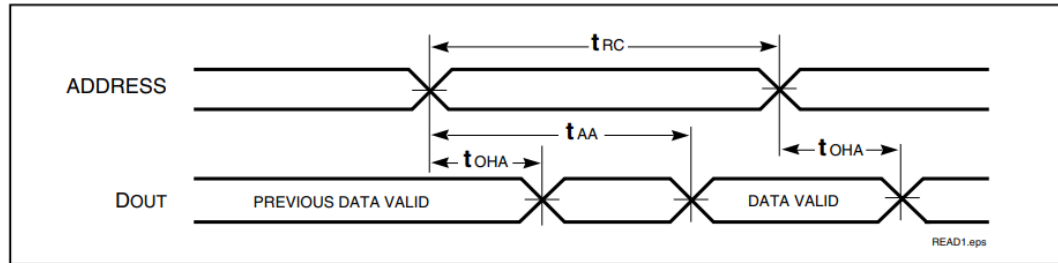
Interfacing to External Memory

- This requires careful inspection of timing diagrams in your memory chip, comparison to microcontroller datasheet
- This is one situation where you should not trust auto-generated code (AI, unknown libraries), as it is often off
- Timing errors mean code might work fine initially, but has errors as temperature change

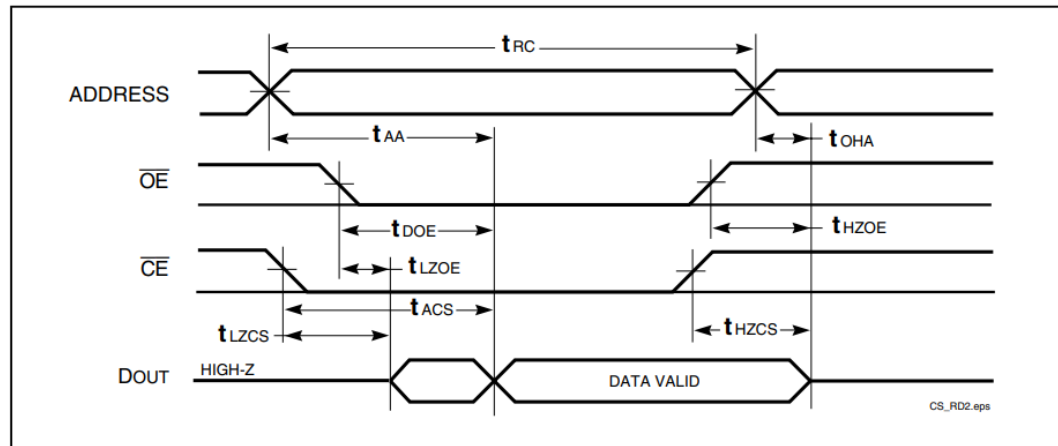
Example of SRAM (Bus) Timing Settings

AC WAVEFORMS

READ CYCLE NO. 1^(1,2)



READ CYCLE NO. 2^(1,3)



Notes:

1. $\overline{WE}$ is HIGH for a Read Cycle.
2. The device is continuously selected. $\overline{OE}$, $\overline{CE}$ = V_{IL} .
3. Address is valid prior to or coincident with $\overline{CE}$ LOW transitions.

<https://www.issi.com/WW/pdf/62-65C256AL.pdf>

READ CYCLE SWITCHING CHARACTERISTICS⁽¹⁾ (Over Oper

Symbol	Parameter	-25 ns	
		Min.	Max.
t_{RC}	Read Cycle Time	25	—
t_{AA}	Address Access Time	—	25
t_{OHA}	Output Hold Time	2	—
t_{ACS}	$\overline{CE}$ Access Time	—	25
t_{DOE}	$\overline{OE}$ Access Time	—	13
$t_{LZOE}^{(2)}$	$\overline{OE}$ to Low-Z Output	0	—
$t_{HZOE}^{(2)}$	$\overline{OE}$ to High-Z Output	0	12
$t_{LZCS}^{(2)}$	$\overline{CE}$ to Low-Z Output	3	—
$t_{HZCS}^{(2)}$	$\overline{CE}$ to High-Z Output	0	12
$t_{PU}^{(3)}$	$\overline{CE}$ to Power-Up	0	—
$t_{PD}^{(3)}$	$\overline{CE}$ to Power-Down	—	20

Notes:

1. Test conditions assume signal transition times of 5 ns or less, timing reference I output loading specified in Figure 1.
2. Tested with the load in Figure 2. Transition is measured ± 500 mV from steady-s
3. Not 100% tested.

Bus Wait-States

- It is common that busses support wait states
- You may need to configure wait states if memory access is slower than the bus is running.

4.5.1 Flash access control register (FLASH_ACR)

Address offset: 0x00

Reset value: 0x0000 0030

Bits 2:0 **LATENCY[2:0]**: Latency

These bits represent the ratio of the HCLK period to the Flash access time.

000: Zero wait state, if $0 < \text{HCLK} \leq 24 \text{ MHz}$

001: One wait state, if $24 \text{ MHz} < \text{HCLK} \leq 48 \text{ MHz}$

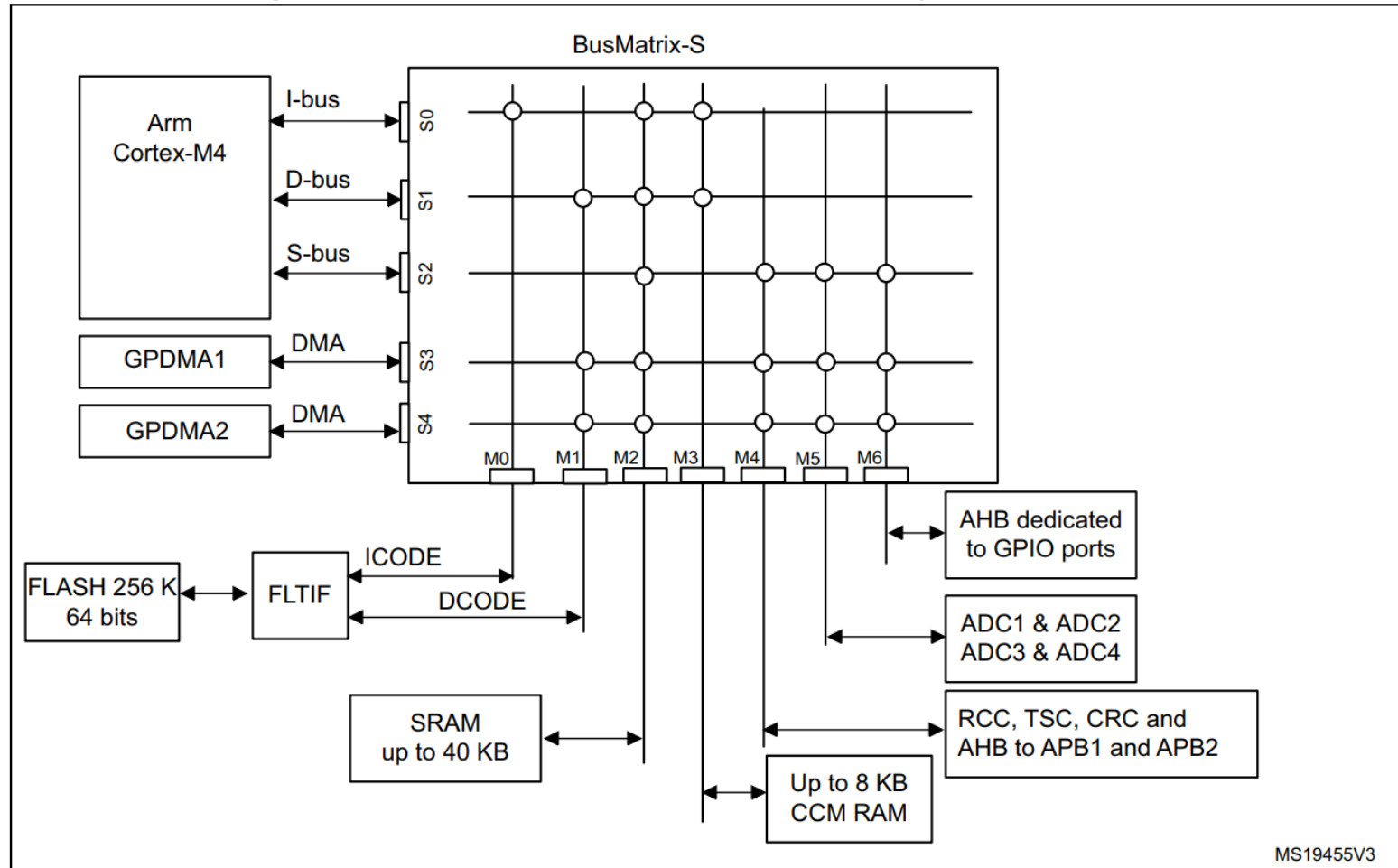
010: Two wait states, if $48 < \text{HCLK} \leq 72 \text{ MHz}$

Busses & Performance

- System performance is often limited by bus performance
- E.g., processor runs at 72 MHz..but if need 2 wait-states between every FLASH access:
 - Naïve reading of flash in sequence slows things down
 - Changes to architecture of memory & caches improve that (future topics)
- Avoiding wait states can be difficult in practice!
- Multiple busses can greatly improve performance.

Careful Memory Management

Figure 1. STM32F303xB/C and STM32F358xC system architecture



e.g., Imagine DMA1 from ADC1 to SRAM

What else can happen at the same time?

Summary: Memory Busses

- Busses are collection of wires, on-chip or off-chip
- Typically we have multiple busses with some buffer/interconnect to provide both high & low performance busses, or to avoid needing to wait for other devices.
- Split transactions and bus arbitration allow busses to be shared among controllers
- Direct Memory Access (DMA) controller is an extremely powerful part of the microcontroller
- May need to manage wait-states & timing to deal with different speed interfaces